DS5003A – Data Engineering

Assignment 4: Searching & Sorting

August 21, 2026

Duration: 2:00 PM – 4:45 PM
Total Questions: 8

Instructions:

- Difficulty increases progressively from Q1 to Q8.
- Follow all constraints specified in each question strictly. Use of prohibited built-in functions will result in loss of marks.
- Write clean, well-commented Python code.

Q1. Pair Sum to Target

Given an array of numbers and a target T, determine the set of two numbers that sum to T.

Example:
Input:
`arr = [3, 5, 6, 7, 8, 10, 12]`, `T = 15`
Output:

```
3 and 12
5 and 10
7 and 8
```

Concept: Two-pointer technique / Hashing.

Q2. Sort an Array of 0's, 1's and 2's

Given an array consisting of only 0's, 1's, and 2's, sort the array in ascending order (all 0's followed by all 1's, followed by all 2's) in $O(n)$ time.

Example:
Input:
`[0, 0, 1, 1, 2, 2, 0, 1, 0, 1, 2, 1, 0, 2]`
Output:
`[0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2]`

Constraint: The solution must run in $O(n)$ time with a single pass over the array.

Q3. Russian Doll Envelopes

You are given a 2D array of integers `envelopes` where `envelopes[i] = [w_i, h_i]` represents the width and height of an envelope. One envelope can fit into another if and only if both the width and height of one envelope are strictly greater than the width and height of the other envelope. Return the maximum number of envelopes you can put one inside the other.

Example:
Input:
`envelopes = [[5,4], [6,4], [6,7], [2,3], [2,1]]`
Output:
`3`
The maximum number of envelopes you can arrange is 3: $[2,3] \Rightarrow [5,4] \Rightarrow [6,7]$ (or $[2,1] \Rightarrow [5,4] \Rightarrow [6,7]$)

Concept: Sorting + Longest Increasing Subsequence (LIS).

Q4. Top K Frequent Elements

Given an integer array `nums` and an integer k, return the k most frequent elements. You may return the answer in any order.

Example:
Input:
`nums = [5, 5, 2, 3, 2, 3, 4, 4, 5, 6, 4, 2, 2]`, `k = 3`
Output:
`2 (4), 4 (3), 5 (3)`

Concept: Frequency counting + sorting / bucket sort by frequency.

Q5. Bucket Sort

Consider an array that you can partition into multiple smaller arrays (buckets). Sort each bucket and concatenate them to get a sorted array. Implement this such that the complexity of the algorithm is $O(n + k)$.

Example:
Input:
`[56, 23, 78, 74, 11, 2, 45, 71, 34]`

Constraint: Overall time complexity must be $O(n + k)$.
Concept: Bucket Sort.

Q6. Sort Records by Multiple Criteria

You are given a list of student/data records:

```
records = [
{"name": "A", "department": "CSE", "score": 82},
{"name": "B", "department": "ECE", "score": 91},
{"name": "C", "department": "CSE", "score": 91},
{"name": "D", "department": "ECE", "score": 75},
{"name": "E", "department": "CSE", "score": 82}
]
```

Write a Python program to sort the records according to:

1. `department` in ascending order.
2. Within each department, `score` in descending order.
3. If both department and score are the same, preserve their original order.

Constraint: Do not use `sort()` or `sorted()`. Implement the sorting logic using Merge Sort.
Concept: Merge Sort + structured records + multiple sorting criteria (stability).

Q7. Top-K Records Using Sorting

A data-processing system stores daily transaction records:

```
transactions = [
("T101", 4500), ("T102", 1200), ("T103", 7800),
("T104", 3200), ("T105", 9500), ("T106", 2100)
]
```

Write a Python program to find the top K transactions by amount. For example, if `K = 3`, the output should contain the three transactions having the highest amounts, in descending order.

Example output:

```
T105 9500
T103 7800
T101 4500
```

Requirements:

- Implement the sorting algorithm yourself.
- Do not use `sort()`, `sorted()`, `max()`, or a priority queue.
- Use your Quick Sort implementation to solve the problem.

Concept: Sorting + practical data processing + Top-K analysis.

Q8. Data Engineering Mini Problem – Batch Processing Analysis

A data pipeline receives records containing a record ID and processing time:

```
records = [
("R101", 45), ("R102", 12), ("R103", 78), ("R104", 34),
("R105", 23), ("R106", 56), ("R107", 12)
]
```

Write a Python program that:

1. Sorts the records by processing time in ascending order using Merge Sort.
2. Accepts a processing-time value from the user.
3. Uses Binary Search to find a record having that processing time.
4. Displays the record ID and processing time if found.
5. Displays `"Record not found"` if no record has that processing time.

Example:
Target processing time = 34

Output:

```
Record ID: R104
Processing time:  34
```

Requirements:

- Implement Merge Sort yourself.
- Implement Binary Search yourself.
- Do not use `sort()`, `sorted()`, `index()`, or `in`.
- The Binary Search must operate on the sorted records.